

Turing Machine

Rafiad Sadat Shahir

June 2026

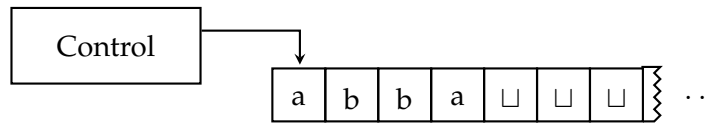
1 Introduction

There exists a much more powerful model of computation, first proposed by Alan Turing in 1936, called the Turing machine. A Turing machine is a much more accurate model of a general purpose computer.

2 Description

A Turing machine consists of three parts:

1. A finite-state control that issues commands,
2. An infinite tape with input and blank cells,
3. A tape head that can read and write a single tape cell.



At each step, the Turing machine

1. Reads and writes a symbol to the tape cell under the tape head,
2. Changes state,
3. Moves the tape head to the left or to the right.

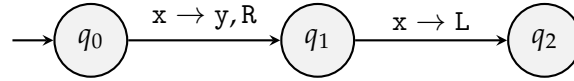
3 Definition

A Turing machine is a 7-tuple $(Q, \Sigma, \Gamma, \delta, q_0, q_{\text{accept}}, q_{\text{reject}})$, where Q , Σ , and Γ are all finite sets, and

1. Q is the set of states,
2. Σ is the input alphabet, not containing the blank symbol $\sqcup$,
3. Γ is the tape alphabet, where $\sqcup \in \Gamma$ and $\Sigma \subseteq \Gamma$,
4. $\delta : Q \times \Gamma \rightarrow Q \times \Gamma \times \{L, R\}$ is the transition function,
5. $q_0 \in Q$ is the start state,
6. $q_{\text{accept}} \in Q$ is the accept state,
7. $q_{\text{reject}} \in Q$ is the reject state, where $q_{\text{reject}} \neq q_{\text{accept}}$.

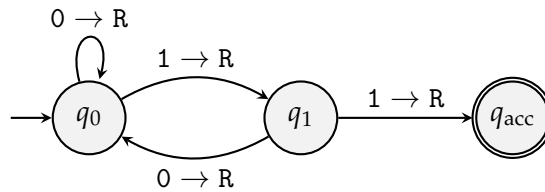
4 Designing Turing Machines

Consider the following state diagram.

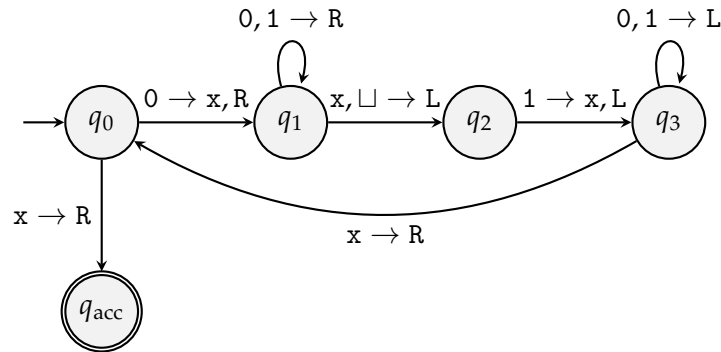


In the diagram, the label $x \rightarrow y, R$ on the transition from q_0 to q_1 signifies that when the machine is in state q_0 and the tape head is reading the symbol x , it 1. Writes the symbol y on the tape, 2. Moves the tape head one cell to the right, and 3. Enters state q_1 . Formally, this transition is represented as $\delta(q_0, x) = (q_1, y, R)$. For simplicity, we use the shorthand notation $x \rightarrow R$ on the transition from q_1 to q_2 to indicate that the machine moves one cell to the right when reading x , without changing the tape symbol. Thus, $\delta(q_1, x) = (q_2, x, L)$. In general, the reject state and all transitions leading to it are omitted for simplicity. Any transition that is not explicitly shown is implicitly assumed to lead to the reject state.

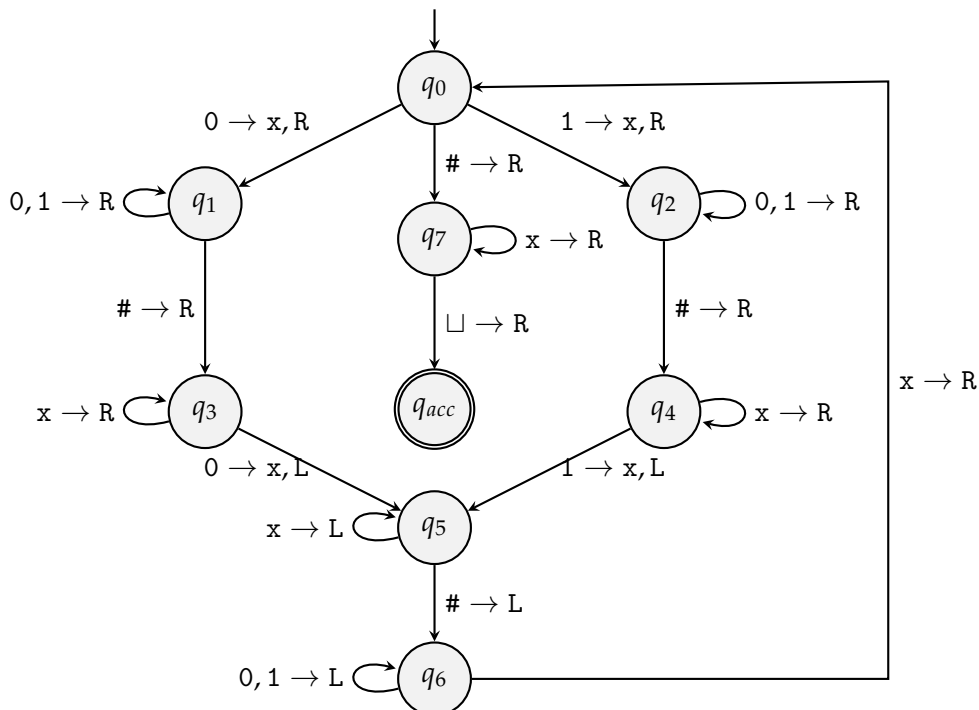
Example 4.1: $L(M) = \{w \in \{0, 1\}^* \mid w \text{ contains } 11\}$



Example 4.2: $L(M) = \{0^n 1^n \mid n > 0\}$



Example 4.3: $L(M) = \{w\#w \mid w \in \{0, 1\}^*\}$



We can formally describe a particular Turing machine by specifying each of its seven components. However, providing all seven components can be cumbersome except for very simple machines. Therefore, Turing machines are often described informally by outlining their operation rather than specifying every transition. The algorithm for the Turing machine M defined above can be summarized as follows:

M = On input string w :

1. Zig-zag across the tape between the corresponding positions on either side of the # symbol. Check whether the symbols at these positions are identical. If they are different, reject. Otherwise, cross off the matched symbols and continue.
2. When all symbols to the left of the # have been crossed off, check whether any uncrossed symbols remain to the right of the #. If any remain, reject; otherwise, accept.

5 Configuration of Turing Machines

As a Turing machine performs a computation, its current state, tape contents, and tape head position change over time. A particular combination of these three components is called a *configuration* of the Turing machine. For a state q and two strings u and v , we write uqv to denote a configuration in which the current state is q , the tape contains the string uv , and the tape head is positioned on the first symbol of v . For example, $011q_701$ represents a configuration in which the tape contains the string 01101 , the current state is q_7 , and the tape head is positioned on the second 0 of the tape.

6 Church-Turing Thesis

The *Church-Turing thesis* is a fundamental principle of computability theory. It states that:

Any effectively calculable function can be computed by a Turing machine.

Alonzo Church formulated this idea using the λ -calculus, whereas Alan Turing expressed the same concept through the Turing machine model. Though not a mathematical theorem, the Church-Turing thesis is widely accepted.

7 Turing Recognizable and Decidable Languages

When a Turing machine starts on an input string, one of the following three outcomes is possible:

1. The machine accepts the input.
2. The machine rejects the input.
3. The machine loops forever (i.e., it never halts).

A Turing machine may fail to accept an input either by explicitly rejecting it or by looping indefinitely. A language accepted by a Turing machine is called a *Turing recognizable* (or *recursively enumerable*) language, and the corresponding Turing machine is called a *recognizer*. If a Turing machine halts on every possible input, then the language it recognizes is called a *Turing decidable* (or *recursive*) language, and the machine is called a *decider*.

8 Notations for Encoding

The input to a Turing machine is always a string. If we wish to provide an object other than a string as input, the object must first be encoded as a string. A Turing machine can then be programmed to decode this string representation. We use the notation $\langle O \rangle$ to denote the encoding of an object O as

a string. The first line of a Turing machine algorithm specifies the expected form of the input. When the input is written as $\langle O \rangle$, the machine implicitly verifies that the input is a valid encoding of the object O . If the encoding is invalid, the machine immediately rejects the input.

9 Decidable Languages

One way to prove that a language is decidable is to construct a Turing machine that decides it. Consider the following language:

$$A_{\text{DFA}} = \{ \langle D, w \rangle \mid D \text{ is a DFA that accepts the input string } w \}.$$

The language A_{DFA} consists of all encodings of a deterministic finite automaton D and an input string w such that D accepts w .

Theorem 9.1: A_{DFA} is decidable.

Proof: The following Turing machine M_{DFA} decides A_{DFA} .

M_{DFA} = On input $\langle D, w \rangle$:

1. Verify that the input has the form $\langle D, w \rangle$, where D is a DFA and w is a string. If not, reject.
2. Simulate the DFA D on the input string w .
3. If the simulation ends in an accepting state, accept; otherwise, reject.

Since a DFA processes exactly one input symbol per step, the simulation always halts after $|w|$ steps. Therefore, M_{DFA} halts on every input and is a *decider*. Consequently, A_{DFA} is decidable.

Note: Throughout these notes, the shorthand “On input $\langle D, w \rangle$ ” implicitly means that the machine has already verified that the input is a valid encoding of a DFA D and a string w .

Theorem 9.2: Let $E_{\text{DFA}} = \{ \langle D \rangle \mid D \text{ is a DFA and } L(D) = \emptyset \}$. Then E_{DFA} is a decidable language.

Proof: A DFA accepts some string if and only if an accepting state is reachable from the start state by following the transitions of the automaton. Therefore, we can determine whether $L(D) = \emptyset$ by checking whether any accepting state is reachable from the start state.

The following Turing machine T decides E_{DFA} .

T = On input $\langle D \rangle$, where D is a DFA:

1. Mark the start state of D .
2. Repeat until no new states are marked:
 - (a) Mark every state that has a transition from a state that is already marked.
3. If no accepting state has been marked, accept; otherwise, reject.

The marking process explores all states reachable from the start state. Since a DFA has a finite number of states, the algorithm always terminates. Therefore, T is a *decider*. Consequently, E_{DFA} is decidable.

10 Universal Turing Machines

Consider the following language:

$$A_{\text{TM}} = \{ \langle M, w \rangle \mid M \text{ is a Turing machine that accepts } w \}.$$

The following Turing machine U recognizes A_{TM} .

U = On input $\langle M, w \rangle$:

1. Simulate the Turing machine M on input w .
2. If M enters its accept state, accept.
3. If M enters its reject state, reject.

The machine U is called a *universal Turing machine*. Proposed by Alan Turing in 1936, it is capable of simulating the computation of any other Turing machine given its description. The concept of the universal Turing machine laid the theoretical foundation for the stored-program computer architecture used in modern computers.

11 The Halting Problem

The *halting problem* asks whether a given Turing machine eventually halts (either by accepting or rejecting) on a given input. The language is defined as follows:

$$HALT_{TM} = \{ \langle M, w \rangle \mid M \text{ halts on input } w \}.$$

Theorem 11.1: $HALT_{TM}$ is undecidable.

Proof: Assume, for contradiction, that there exists a Turing machine H that decides $HALT_{TM}$.
 $H =$ On input $\langle M, w \rangle$:

1. Simulate M on input w .
2. If M halts, accept.
3. If M runs forever, reject.

Using H , construct another Turing machine D .
 $D =$ On input $\langle M \rangle$:

1. Run H on input $\langle M, \langle M \rangle \rangle$.
2. If H rejects, halt.
3. If H accepts, loop forever.

Thus,

$$D(\langle M \rangle) = \begin{cases} \text{halts,} & \text{if } M \text{ loops forever on } \langle M \rangle, \\ \text{loops forever,} & \text{if } M \text{ halts on } \langle M \rangle. \end{cases}$$

When we run D on its own description $\langle D \rangle$, we get:

$$D(\langle D \rangle) = \begin{cases} \text{halts,} & \text{if } D \text{ loops forever,} \\ \text{loops forever,} & \text{if } D \text{ halts.} \end{cases}$$

However, this contradicts our assumption that H solves the halting problem for any Turing machine and its input. Therefore, our assumption must be wrong. Hence, H must not exist and $HALT_{TM}$ must be undecidable.